SudoSLVRR: A Multithreaded Multi-Colony Ant Optimization with Ring and Random Communication Topologies as Sudoku Solver

John Michael E. Jabuen, Gian Myrl Renomeron, and
John Paul T. Yusiong

Division of Natural Science and Mathematics
University of the Philippines Tacloban College
Magsaysay Boulevard, Tacloban City 6500, Leyte, Philippines

Abstract. A popular logic-based combinatorial problem known as Sudoku is computationally expensive because of its interdependent constraints. From traditional solvers to heuristics approaches, although effective when it comes to smaller puzzles, faces difficulties with more complex puzzles. This paper presents SudoSLVRR, a parallel multi-colony ant optimization as sudoku solver that addresses scalability limitations of the existing solvers. This framework combines constraint propagation and multi-colony ant optimization with dynamic collaborative mechanism (DCM-ACO) inside the multithreaded environment. Each multi-colony ant is executed inside each thread where threads exchange their iteration-best solution and best-so-far solution in ring and random communication topologies, respectively. The three-source pheromone update is performed by each thread based on the received information. Experimental results on 9×9, 16×16, and 25×25 Sudoku puzzles shows that the proposed framework was able to achieve higher solution success rate and less solution completion time even in larger Sudoku puzzles when compared to single-colony ant and non-parallel multi-colony ant solvers.

Keywords: Constraint Satisfaction Problem · Swarm Intelligence · Ant Colony Optimization · Dynamic Collaborative Mechanism · Parallel Optimization

1 Introduction

Sudoku is one of those puzzles that looks deceptively simple: fill an $n \times n$ grid with the digits 1 through n so that no row, column, or subgrid repeats a value. Behind that simplicity, though, lies a rich constraint satisfaction problem, which is why it has become such a popular testbed in the optimization literature [3, 10, 5, 8]. The earliest solvers leaned on deterministic backtracking [12, 15, 17, 11] or tried to mimic the pencil-and-paper strategies that human players use [13, 17]. Both work fine for small grids, but they hit a wall once the puzzle dimensions start growing. That limitation motivated a wave of swarm-based metaheuristics—Artificial Bee Colony [6], Particle Swarm Optimization [9, 16], Ant Colony Optimization (ACO) [3]—which are better equipped to navigate large search spaces.

Among these, ACO stood out as especially promising, particularly when paired with constraint propagation (CP) to prune infeasible regions early [12, 3].

Still, there is a persistent gap in the literature. Most solvers lean into either constraint-aware search or parallel exploration, but rarely both at once. Hybrid CP–ACO methods do a good job of shrinking the feasible space, yet they typically run a single colony on a single thread, which makes them prone to pheromone stagnation as grids get larger [3]. Multi-colony designs like DCM-ACO address part of this by using entropy-based cooperation across heterogeneous colonies to keep a healthy tension between convergence and diversity—but they still operate within one processing context, with no way to coordinate across threads [4]. On the parallel side, approaches such as RMACO spread colonies over multiple processors using structured communication topologies, but they skip over the question of colony-role heterogeneity and do not actively regulate diversity within each colony [2]. The upshot is that none of these methods reliably sustain both convergence and diversity as puzzle complexity scales up.

This paper introduces **SudoSLVRR** to bridge that gap. SudoSLVRR is a multithreaded Sudoku solver that nests DCM-ACO inside a communication-aware parallel architecture. Each thread hosts a full complement of heterogeneous colonies; internally, their pheromone landscapes are coordinated through entropy-regulated cooperation, while externally, threads exchange solutions via ring-topology and random-matching protocols. The result is a solver that can balance exploration and exploitation within each thread and, at the same time, share useful information across threads—yielding robust performance on 9×9, 16×16, and 25×25 puzzles. Lloyd and Amos [3] pair constraint propagation with Ant Colony System (ACS) [14]: CP first strips out invalid cell-value assignments, and then ants build solutions steered by a global pheromone matrix. To keep the pheromone landscape from going stale, they introduce a Best Value Evaporation (BVE) operator that gradually decays deposited pheromone over time. The approach works well up to a point, but because it relies on a single colony, the convergence it gains tends to come at the cost of search diversity—a trade-off that becomes increasingly painful on harder, larger puzzles.

Mo et al. [4] tackle that diversity problem head-on with DCM-ACO, a heterogeneous framework that runs two ACS sub-colonies alongside one Max–Min Ant System (MMAS) [14] sub-colony. The key idea is a cooperative game strategy that redistributes pheromone income across the ACS colonies based on both solution quality and information entropy. When an ACS colony starts to stagnate, a pheromone fusion mechanism kicks in, blending its matrix with the MMAS colony's pheromone landscape to inject fresh information. On top of that, a public path recommendation feeds commonly selected assignments from the ACS colonies into MMAS, giving it a convergence boost. The net effect is a nice balance between fast exploitation through ACS and broad exploration through MMAS. The catch, however, is that DCM-ACO runs entirely within a single process—there is simply no mechanism for spreading the computation across multiple threads.

Yang and Fang [2] come at the problem from the parallelism angle with RMACO, a parallel ACO distributed over MPI where each processor hosts its own colony. Colonies share information through a dual topology: a fixed ring passes along iteration-best solutions, while a randomly matched partner supplies best-so-far solutions. The exchange schedule is adaptive—longer intervals early on let each colony explore independently, then shorter intervals later help the whole system zero in on good regions faster. This three-source pheromone update gives each colony a broader view of the search landscape, but RMACO uses homogeneous colonies throughout and has no entropy-driven mechanism for regulating diversity. It is worth noting that earlier parallel designs had their own shortcomings: Parallel Independent Runs and fine-grained ant-level parallelism [7] suffer from redundant exploration due to absent information sharing and synchronization overhead on global pheromone structures, respectively.

SudoSLVRR draws on all three of these threads. It brings together CP-guided ACO search, heterogeneous multi-colony cooperation with entropy-based pheromone regulation, and structured dual-topology communication—all within a multithreaded framework designed to handle Sudoku puzzles of varying sizes.

2 SudoSLVRR: Algorithm Design and Structure

2.1 Initial Constraint Propagation

Sudoku has a large solution space and navigating through it requires time. Constraint propagation by its heuristic characteristics narrowed down the solution space. This preprocessing mechanism allows the ants from each thread to prevent wasting CPU time in trying invalid cell-value assignments.

CP removes invalid assignments using the fixed values (clues) on the puzzle. Every cell has its own candidate values and belongs to a specific row, column, and subgrid. Rule 1 collects all fixed values from a certain row, column, and subgrid and removes this set of fixed values from the candidate values of the cell's peers (cells that reside to the same row, column, and subgrid). If there is only one remaining value, CP fixes that value to that cell. The second rule is responsible for fixing hidden singles to a cell. The cell is known to have a hidden single when there is only one remaining value on its candidate values after removing all the candidate values of its peers. This process is recursively done until no further deduction can be made. By doing this, the puzzle is reduced and the search space is narrowed as the number of fixed cells has increased.

Algorithm 1: SudoSLVRR (Part 1)

```
Input: Sudoku puzzle P
Input: Parameters: τ0, E(p)*, Con*, timeout
Output: Solved Sudoku puzzle
read puzzle P;
forall cells with fixed values do
    propagate constraints;
                                        // Initial constraint propagation
forall colony do
    initialize pheromone matrix τij ← τ0;
iter ← 0;
while puzzle is not solved and timeout not reached do
    forall threads in parallel do
        give each ant a local copy of the puzzle;
        assign each ant to a different cell;
        for number of cells do
            for each colony do
                for each ant do
                    if current cell value not fixed then
                        choose value from current cell's value set;
                        fix cell value;
                        propagate constraints;
                        update local pheromone using Equation (2);
                    move to next cell;
        for each colony do
            find the iteration-best in thread using Equation (3);
            calculate the pheromone to add, Δτ, using Equation (4);
            if Δτ iteration-best > global-iteration-best pheromone then
                set global-iteration-best solution to iteration-best solution;
                set Δτ global-iteration-best to Δτ iteration-best;
            if Δτ iteration-best > Δτ best-so-far then
                set best-so-far to iteration-best;
                set Δτ best-so-far to Δτ iteration-best;
            if Δτ best-so-far > Δτ global-best-so-far then
                set global-best-so-far to Δτ best-so-far;
                set global-best-so-far to best-so-far;
        // Apply Dynamic Collaborative Mechanism (DCM-ACO)
        divide ACS colonies into low-entropy and high-entropy groups using
          E(p)*;
        if low-entropy ACS group is not empty then
            apply pheromone fusion with MMAS colony using Equations
              (15)–(16);
        if high-entropy ACS group is not empty then
            apply cooperative game theory using Equations (9)–(14);
            perform global pheromone update using Equation (5);
            perform Best Value Evaporation using Equation (6);
        calculate convergence speed for the MMAS colony;
        if convergence speed < Con* then
            apply public path recommendation using Equations (17)–(19);
    // Algorithm continues on the next page...
```

Algorithm 1: SudoSLVRR (continued)

```
while puzzle is not solved and timeout not reached (continued) do
    // ...continued from previous page
    if communication interval is reached then
        synchronize all threads at barrier;
        generate random matching array;
        exchange iteration-best solutions using ring topology;
        exchange best-so-far solutions using random topology;
        perform global pheromone update using Equations (20)–(21);
    else
        perform global pheromone update for the MMAS colony using
          Equation (5);
    increment iter;
```

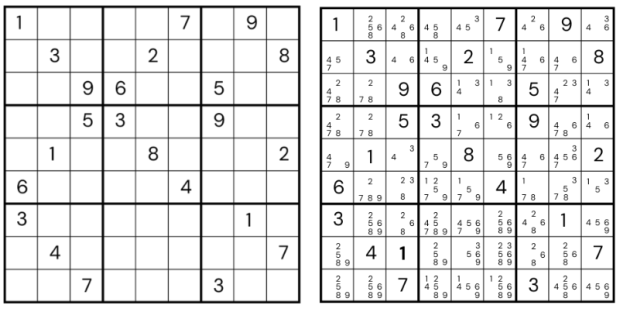

Fig. 1. AI Escargot Sudoku puzzle (left), and the result after CP (right). The value sets are represented as strings of allowable digit for a certain cell. For example, '2568' represents the set of values 2,5,6,8.

2.2 DCM-ACO

The multi-colony ant optimization for SudoSLVRR is composed of ACS and MMAS colonies. Ant Colony System (ACS) is known for its rapid convergence while Min-Max Ant System has its strong exploratory characteristics. Thus, the two colonies balance the exploitation and exploration mechanism of the algorithm by sharing pheromone information.

Pheromone Matrix Initialization Since the algorithm is composed of multiple colonies, it maintains a pheromone matrix for each colony τij where i represents the index of the cell and j represents the candidate value from the candidate set $\{1, 2, 3, \ldots, d\}$. For a 9×9 Sudoku puzzle, $d = 9$ denotes the dimension of the Sudoku puzzle. All the values in the pheromone matrix are initialized

to the same value given by

$$\tau_0 = \frac{1}{c} \tag{1}$$

where $c = d^2$ represents the total number of cells in the puzzle.

Ant Solution Construction The resulting board from the initial constraint propagation is given to each ant to have their own local copy of the board (Line 8). Each ant starts at a random starting cell (Line 9) and moves to the next cell following the left-to-right direction until it goes back to the starting cell (Line 10).

Ant Decision Rule At each visited cell, an ant must choose a value for this cell (Line 14). The greediness of the ant in the ACS colony is determined by the parameter q_0. When a random number $r \in [0, 1]$ is less than q_0, then the ant acts greedily, choosing the value with the highest pheromone concentration from the candidate values of the cell. Otherwise, the ant performs roulette wheel selection where probabilities of choosing a value is determined by its pheromone concentration. This selection gives the value a chance even with less pheromone concentration to be fixed for the cell. The ants in MMAS colony always perform roulette wheel selection highlighting its exploratory characteristics. Therefore, the q_0 in this colony is set to 0.

Propagate Constraints Propagate Constraints After the ant decides what to fix into the cell (Line 15), it calls the CP to propagate the changes (Line 16). The CP, again, performs the previously discussed rules in removing invalid cell-value assignments. This propagation ensures that every step the ants make is valid while narrowing down the solution space.

Local Pheromone Update To reduce the probability of the certain value of being chosen again, ants perform local pheromone update (Line 17). This update reduces the pheromone concentration of the previously chosen value, encouraging the subsequent ants to explore other values for the cell. The new pheromone concentration is given by:

$$\tau_{ij} \leftarrow (1 - \xi) \cdot \tau_{ij} + \xi \cdot \tau_0 \tag{2}$$

where $\xi = 0.1$. It is important to note that MMAS colony does not perform local pheromone update. MMAS colony preserves each pheromone concentration but implements a pheromone bound.

Solution Evaluation and Iteration-Best Selection After all ants have constructed their solution, the algorithm quantifies the quality of the solution produced by each ant. The iteration-best ant is the ant who has the highest number of filled cells (Line 18). At each iteration, every ant n of the m ants records the

total number of cells it was able to fill, f_n. The algorithm compares these scores and sets the f_{best} with the maximum number of filled cells:

$$f_{\text{best}} = \max_{n \in \{1 \ldots m\}} f_n. \tag{3}$$

Pheromone Contribution Calculation The algorithm calculates the pheromone contribution of the iteration-best solution according to

$$\Delta\tau = \frac{c}{c - f_{\text{best}}} \tag{4}$$

The value of the computed $\Delta\tau$ is compared with the current contribution of the best-so-far, $\Delta\tau_{best}$, (with an initial value of 0). If τ is greater than the $\Delta\tau_{best}$, then the algorithm set $\Delta\tau_{\text{best}} \leftarrow \Delta\tau$ Followed by replacing the current best-so-far solution with the iteration-best solution.

Standard Global Pheromone Update To reinforce the most successful path so far, the algorithm updates the pheromone matrix by adding the pheromone contribution to the cell-value assignments found in the best-so-far solution:

$$\tau_{ij}(t+1) = (1-\rho) \cdot \tau_{ij}(t) + \rho \cdot \Delta\tau_{\text{best}} \tag{5}$$

where $\rho = 0.9$ is the evaporation rate and $\Delta\tau_{\text{best}}$ is the calculated pheromone contribution from the best-so-far solution.

However, this update might cause the algorithm to get stuck to the same solution forever. To prevent this from happening, best value evaporation is executed:

$$\Delta\tau_{\text{best}} \leftarrow \Delta\tau_{\text{best}} \cdot (1 - \rho_{\text{BVE}}) \tag{6}$$

where $\rho_{\text{BVE}} = 0.005$ is a predefined parameter that dictates the evaporation rate of the best pheromone value.

Best Value Evaporation (BVE) is another update that MMAS colony does not perform. Aside from its 100% random value selection to prevent stagnation, it implements strict pheromone bound (upper and lower bound):

$$\tau_{\max} = \frac{1}{\rho} \cdot (\Delta\tau) \tag{7}$$

$$\tau_{\min} = \frac{\tau_{\max}}{2 \cdot N} \tag{8}$$

where ρ is the evaporation rate and N is the total number of cells in the puzzle (for example in 9×9 Sudoku puzzle, $N = 81$).

The maximum pheromone concentration a cell-value assignment can accumulate is only up to $\tau_{\max}$. Conversely, $\tau_{\min}$ dictates the minimum pheromone concentration a cell-value assignment can accumulate. These global bounds prevent the algorithm from converging early to frequent cell-value assignments.

Dynamic Collaborative Mechanism

A. **Cooperative Game Theory** This collaboration is applied only to ACS colonies where best pheromone value from each of these colonies are added to get the total pheromone income b:

$$b = \sum_{i=1}^{k} \Delta\tau_{ACS}^{i} \tag{9}$$

This total pheromone income is then redistributed to ACS colonies based on their contribution which depends on the solution quality of the colony's best-so-far solution and its diversity. The diversity of the colony is being quantified using information entropy:

$$P_i = \frac{k_i}{K} \tag{10}$$

$$E(t) = -\sum_{i=1}^{U} P_i \log P_i \tag{11}$$

where K is the number of ants in the ACS colony, U is the number of unique solutions generated by ants, and k_i represents the number of ants that generated the $i-$th unique solution. This calculation shows that the higher the entropy, the higher the diversity of the colony.
The contribution of the i-th ACS colony is given by:

$$\text{Contribution}_i = \text{sol}_i \cdot H(p_i) \tag{12}$$

where $\text{sol}_i = \frac{\text{length}_{\min}}{\text{length}_i}$ and $H(p_i) = \frac{E(p_i)}{E(p_{\max})}$. In computing the *length* for the solution quality of the ACS colony, we calculated the number of unfilled cells from the best-so-far solution of the ACS colony by subtracting the number of filled cells from the total number of cells. After computing the information entropy of all the ACS colonies, we set $E(p_{\max})$ to the maximum information entropy.
After the algorithm computes the contribution of each ACS colony, it begins redistributing the total pheromone income b according to their contribution.

$$\pi_i = \frac{\text{Contribution}_i}{\sum_{i=1}^{k} \text{Contribution}_i} \tag{13}$$

$$\Delta\tau_i^{\text{new}} = \pi_i \cdot b \tag{14}$$

where $\Delta\tau_i^{\text{new}}$ is the new pheromone value of the i-th ACS colony. This new pheromone value, $\Delta\tau_i^{\text{new}}$, replaces the best pheromone value of the ACS colony and is now the value used for the standard global pheromone update.

B. **Pheromone Fusion** As ACS is known for its aggressive exploitation, it is prone to early stagnation. To help the stagnating ACS colony recover, the algorithm injects MMAS pheromone information to the stagnating ACS colony. ACS colony is considered to be stagnating when its computed entropy, $E(p_i)$, falls below the predefined entropy threshold $E^*(P)$, denoted as $E(p_i) < E^*(P)$. To determine how strong the pheromone fusion is, we calculate the weight factor ω_i:

$$\omega_i = \frac{E(P)^i_{ACS}}{E(P)^i_{ACS} + E(P)_{MMAS}} \tag{15}$$

$$ph^i_{ACS} \leftarrow (1 - \omega_i) \cdot ph^i_{ACS} + \omega_i \cdot ph_{MMAS} \tag{16}$$

This mechanism updates the entire pheromone matrix of the stagnating ACS colony with the injected diversity of the MMAS colony.

C. **Public Path Recommendation** Because of the strong exploratory characteristics of the MMAS colony, it suffers from slow convergence. To assist the MMAS colony to convergence, common cell-value assignments are identified from the ACS colonies.

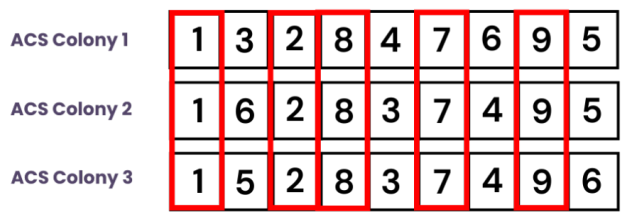

Fig. 2. Common cell-value assignments (public path) for the first row are highlighted with red box.

This mechanism is triggered when the computed convergence rate (con_t) of the MMAS colony falls below the predefined convergence threshold (con^*), denoted as ($con_t < con^*$).

$$con_t = \frac{iter_{opt}}{iter_t} \tag{17}$$

where ($iter_{opt}$) refers to the iteration number where the current optimal solution was found and ($iter_t$) refers to the current iteration.

$$\tau_{public} = \frac{1}{N} \cdot e^{-iter} \tag{18}$$

where N is the total number of cells in the puzzle and (τ_{public}) is the amount of pheromone to be added. The calculated pheromone value to add is added to the current pheromone value of the common paths (i, j) identified from

the ACS colonies. This strategy allows the MMAS colony to explore into this region of the solution space.

$$\tau_{i,j}^{public} \leftarrow \tau_{i,j}^{public} + \tau_{public} \tag{19}$$

where $\tau_{i,j}^{public}$ represents the pheromone value of the cell-value assignments (i, j) in the MMAS pheromone matrix.

2.3 Thread Communication

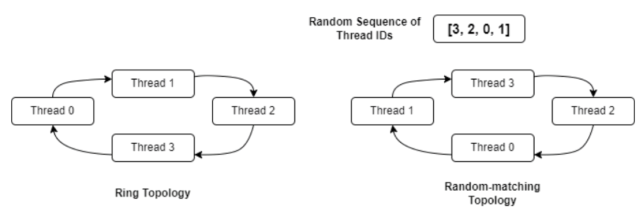

Fig. 3. Ring and Random-matching Communication Topologies

Adapted from [2],this solver implements two communication topologies as shown in Figure3. It is important to note here that the master thread does not collect solutions from other threads but only generates an array containing a random sequence of the thread IDs. In this communication, pheromone information are shared but not through exchanging the entire pheromone matrix as it increases communication overhead between threads according to Yang and Fang [2], instead only the path information is exchanged. Each thread waits at the barrier when the communication interval is reached. Each one of them receives two solutions from other threads. It then uses these to do the three-source communication global pheromone update.

$$\Delta\tau_{ij} = \Delta\tau_{ij}^{(1)} + \Delta\tau_{ij}^{(2)} + \Delta\tau_{ij}^{(3)} \tag{20}$$

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \rho\Delta\tau_{ij} \tag{21}$$

where ρ is the pheromone evaporation rate, and τ_{ij} is the pheromone value of the value j to cell i. The total pheromone deposit $\Delta\tau_{ij}$ consists of three components:

1. $\Delta\tau_{ij}^{(1)}$ — the thread's own iteration-best solution;
2. $\Delta\tau_{ij}^{(2)}$ — the received iteration-best solution from ring topology; and
3. $\Delta\tau_{ij}^{(3)}$ — the received best-so-far solution from a randomly-matched neighbor.

Only the MMAS colony performs this update as external pheromone reinforcement to the ACS colonies may disrupt their true diversity thus disrupting also the cooperative game strategy. Although ACS colonies are not directly using this pheromone information, they are still indirectly influenced through pheromone fusion.

3 Results and Discussion

To evaluate the performance of SudoSLVRR (MCD), it was compared with the baseline solvers namely Constraint Propagation and Ant Colony System and Constraint Propagation and Multi-Colony ant Optimization with Dynamic Collaborative Mechanism (CD) using four performance metrics. A total of 45 puzzles, 15 instances each for 9×9, 16×16, and 25×25 Sudoku puzzle, were used and can be downloaded here: https://zenodo.org/records/18812686. Different sizes of Sudoku puzzles are used specifically 9×9, 16×16, and 25×25 with 100 runs per instance in each puzzle size to accurately measure the success rate, average completion time, and average number of iterations.

From the experimental results, we can see that SudoSLVRR (MCD) was able to achieve a higher success rate compared to CA and CD solvers. Although CP-DCM-ACO (CD) solver has heterogeneous colonies that balances exploration and exploitation, it still wasn't able to find a solution when solving high-order Sudoku puzzles. This limitation shows that a further exploration mechanism is needed when solving high-order Sudoku puzzle as the solution space grows larger when the puzzle size increases.

The parallel search allows the SudoSLVRR (MCD) to cover more regions in the solution space thus achieving a higher success rate than the single-thread solvers. The communication between threads also helps the threads not to visit the same region by sharing pheromone information thus speeding up the search for a solution.

4 Conclusions and Future Work

The experimental results showed that developing the SudoSLVRR solver helps speed up the search in very large solution space. The proposed solver achieved near-perfect solution success rate and showed a huge reduction in execution time. Placing the integration of constraint propagation and dynamic collaborative mechanism of multi-colony ant optimization inside each thread that communicates periodically creates a robust framework that reduces exponential time increase when faced with complex constraint satisfaction problems (CSP).

Future work may explore adapting the multithreaded multi-colony framework to highly constrained problems like university timetabling and job-shop scheduling.

Table 1. Success rates (number of solved instances out of 100 runs) together with solution times (reported as mean and standard deviation over successful runs) for the Sudoku instances. The evaluated methods include Constraint Propagation and Ant Colony System (CA), CP-DCM-ACO (CD), and the multithreaded variant of CP-DCM-ACO (MCD). The size parameter indicates the puzzle order: 3 corresponds to 9×9, 4 to 16×16, and 5 to 25×25. Best values are highlighted in bold (time and iterations obtained by algorithms with the highest success rate are compared).

Size	(#)	Success (%)			Solution Time (s)			Iterations		
		CA	CD	MCD	CA	CD	MCD	CA	CD	MCD
3	1	100	100	100	0.11570 ± 0.37344	0.02908 ± 0.02725	$\mathbf{0.01744 \pm 0.01471}$	114.62	44.31	**18.29**
3	2	100	100	100	0.09407 ± 0.08255	0.07581 ± 0.06920	$\mathbf{0.03493 \pm 0.02984}$	84.05	105.82	**34.54**
3	3	100	100	100	0.18462 ± 0.14311	0.15563 ± 0.12174	$\mathbf{0.07195 \pm 0.07092}$	210.11	240.54	**77.29**
3	4	100	100	100	0.09273 ± 0.07672	0.04091 ± 0.04320	$\mathbf{0.02017 \pm 0.02231}$	93.49	64.29	**22.02**
3	5	100	100	100	0.10360 ± 0.11156	0.04594 ± 0.04567	$\mathbf{0.02098 \pm 0.02891}$	102.77	71.81	**22.53**
3	6	100	100	100	0.13264 ± 0.11377	0.09507 ± 0.08579	$\mathbf{0.04666 \pm 0.04423}$	119.37	140.75	**48.36**
3	7	100	100	100	0.09941 ± 0.08250	0.05455 ± 0.04938	$\mathbf{0.03354 \pm 0.02426}$	96.95	86.09	**36.97**
3	8	100	100	100	0.06131 ± 0.05719	0.03178 ± 0.02937	$\mathbf{0.01644 \pm 0.01328}$	58.47	49.15	**17.36**
3	9	100	100	100	0.13674 ± 0.12042	0.11317 ± 0.10481	$\mathbf{0.05119 \pm 0.06579}$	133.54	172.12	**54.04**
3	10	100	100	100	0.11385 ± 0.09377	0.05270 ± 0.04901	$\mathbf{0.03119 \pm 0.02647}$	121.46	87.58	**35.23**
3	11	100	100	100	0.14536 ± 0.13257	0.06470 ± 0.05765	$\mathbf{0.04280 \pm 0.03989}$	138.84	97.77	**46.13**
3	12	100	100	100	0.09800 ± 0.08065	0.07007 ± 0.06278	$\mathbf{0.03973 \pm 0.03313}$	105.07	117.32	**46.16**
3	13	100	100	100	0.07120 ± 0.05234	0.04258 ± 0.03504	$\mathbf{0.02305 \pm 0.01946}$	82.26	78.68	**29.26**
3	14	100	100	100	0.13179 ± 0.10379	0.06788 ± 0.07463	$\mathbf{0.03408 \pm 0.03313}$	134.91	110.93	**38.83**
3	15	100	100	100	0.09368 ± 0.07832	0.03953 ± 0.03854	$\mathbf{0.02386 \pm 0.02190}$	101.01	67.04	**28.33**
4	1	68	100	100	0.57771 ± 1.91163	$\mathbf{0.00006 \pm 0.00011}$	0.01618 ± 0.01019	130.10	12.23	**3.21**
4	2	72	100	100	2.12121 ± 3.74344	$\mathbf{0.00110 \pm 0.00289}$	0.07685 ± 0.10341	432.04	186.93	**16.13**
4	3	69	100	100	1.02604 ± 2.42425	$\mathbf{0.00019 \pm 0.00030}$	0.04976 ± 0.07943	176.58	35.13	**9.18**
4	4	86	100	100	0.50857 ± 1.43724	$\mathbf{0.00005 \pm 0.00006}$	0.01820 ± 0.00705	58.31	9.85	**4.24**
4	5	83	100	100	0.38002 ± 2.15321	$\mathbf{0.00009 \pm 0.00025}$	0.02446 ± 0.01083	82.00	18.01	**4.74**
4	6	87	100	100	0.16247 ± 0.62430	$\mathbf{0.00003 \pm 0.00004}$	0.01542 ± 0.00638	32.01	5.87	**3.06**
4	7	85	100	100	2.03609 ± 4.08408	$\mathbf{0.00206 \pm 0.00466}$	0.10414 ± 0.22509	382.04	330.55	**19.76**
4	8	90	100	100	5.32392 ± 5.34979	$\mathbf{0.00409 \pm 0.00575}$	0.41829 ± 0.74387	949.22	642.49	**76.58**
4	9	92	100	100	0.33880 ± 1.65168	$\mathbf{0.00004 \pm 0.00005}$	0.01892 ± 0.00653	67.82	6.93	**3.96**
4	10	85	100	100	1.63991 ± 3.18775	$\mathbf{0.00010 \pm 0.00019}$	0.03036 ± 0.02392	358.76	18.79	**5.72**
4	11	88	100	100	1.86170 ± 4.00618	$\mathbf{0.00086 \pm 0.00174}$	0.11610 ± 0.17965	417.07	145.30	**22.40**
4	12	85	100	100	4.50037 ± 5.47277	$\mathbf{0.00132 \pm 0.00238}$	0.22387 ± 0.42108	343.75	211.00	**37.75**
4	13	85	100	100	1.75677 ± 3.46634	$\mathbf{0.00077 \pm 0.00089}$	0.18660 ± 0.24480	446.88	136.10	**41.36**
4	14	86	100	100	0.57584 ± 1.94727	$\mathbf{0.00028 \pm 0.00053}$	0.04304 ± 0.07132	96.30	46.30	**8.66**
4	15	90	100	100	4.39263 ± 4.60514	$\mathbf{0.00245 \pm 0.00289}$	0.50979 ± 0.81102	720.74	384.89	**97.09**
5	1	29	89	**97**	11.67848 ± 27.03170	21.53801 ± 32.42817	$\mathbf{14.28408 \pm 25.96476}$	678.72	2213.56	**1057.89**
5	2	29	72	**98**	18.84435 ± 22.20281	29.04373 ± 28.17707	$\mathbf{18.06169 \pm 20.26154}$	613.90	1752.89	**797.26**
5	3	16	46	**83**	18.11757 ± 15.55764	32.14994 ± 30.42519	$\mathbf{31.56935 \pm 30.51672}$	678.25	1998.20	**1472.30**
5	4	37	86	**93**	19.54744 ± 21.96407	29.02063 ± 28.89732	$\mathbf{20.90431 \pm 21.24425}$	624.08	1754.11	**461.09**
5	5	33	87	**100**	21.69408 ± 23.04694	20.50921 ± 23.26286	$\mathbf{11.24212 \pm 14.66900}$	702.18	1285.30	**450.15**
5	6	34	88	**99**	15.88107 ± 20.30579	24.30854 ± 25.68494	$\mathbf{14.80719 \pm 14.02413}$	506.97	1518.26	**678.98**
5	7	84	100	100	11.93498 ± 20.64542	5.46017 ± 6.62348	$\mathbf{3.31359 \pm 3.54204}$	369.88	319.96	**133.62**
5	8	41	95	**98**	6.57945 ± 17.29177	13.70752 ± 26.84342	$\mathbf{1.74019 \pm 8.07977}$	350.41	1232.26	**113.18**
5	9	41	89	**99**	17.74087 ± 30.91254	15.50208 ± 25.43841	$\mathbf{3.26660 \pm 4.18599}$	921.05	1342.02	**209.09**
5	10	42	91	**100**	22.03315 ± 27.03095	32.78987 ± 31.24208	$\mathbf{16.57041 \pm 15.25132}$	767.98	2033.10	**682.50**
5	11	44	90	**100**	23.46806 ± 33.38732	18.95366 ± 19.69135	$\mathbf{11.27289 \pm 12.15516}$	823.48	1226.94	**503.21**
5	12	52	91	**99**	10.41762 ± 24.07106	7.54007 ± 15.28184	$\mathbf{2.53927 \pm 7.87149}$	701.71	831.97	**204.04**
5	13	47	88	**100**	29.42990 ± 31.31428	24.56157 ± 25.86809	$\mathbf{11.95545 \pm 10.10140}$	929.81	1392.84	**509.42**
5	14	52	82	**100**	15.06953 ± 21.06873	22.70938 ± 26.69476	$\mathbf{8.27965 \pm 7.75721}$	590.67	1329.77	**367.66**
5	15	59	100	100	7.59640 ± 18.07987	8.46023 ± 14.13509	$\mathbf{1.36241 \pm 2.06105}$	454.29	847.91	**100.44**

References

1. M. Dorigo, "Optimization, Learning and Natural Algorithms," Ph.D. Dissertation, Politecnico di Milano, 1992.
2. Q. Yang and L. Fang, "RMACO: A Randomly Matched Parallel Ant Colony Optimization," *World Wide Web*, vol. 19, no. 6, pp. 1009–1022, 2015. [Online]. Available: https://doi.org/10.1007/s11280-015-0373-2
3. H. Lloyd and M. Amos, "Solving Sudoku with Ant Colony Optimization," *IEEE Transactions on Games*, vol. 12, no. 3, pp. 302–313, 2020. [Online]. Available: https://doi.org/10.1109/TG.2020.2969827
4. Y. Mo, Z. Tang, and L. Zhao, "Multi-Colony Ant Optimization with Dynamic Collaborative Mechanism and Cooperative Game," *Information Sciences*, vol. 608, pp. 892–908, 2022. [Online]. Available: https://doi.org/10.1016/j.ins.2022.06.055
5. Y. Manyam, S. Sundaram, and A. Sahay, "Sudoku Solver Algorithms: A Comparative Analysis," in *2024 First International Conference for Women in Computing (InCoWoCo)*, IEEE, 2024, pp. 1–9. [Online]. Available: https://doi.org/10.1109/InCoWoCo58689.2024.10443752
6. J. A. Pacurib, G. M. M. Seno, and J. P. T. Yusiong, "Solving Sudoku Puzzles Using Improved Artificial Bee Colony Algorithm," in *2009 Fourth International Conference on Innovative Computing, Information and Control (ICICIC)*, 2009, pp. 885–888. [Online]. Available: https://doi.org/10.1109/ICICIC.2009.334
7. M. Randall and A. Lewis, "A Parallel Implementation of Ant Colony Optimization," *Journal of Parallel and Distributed Computing*, vol. 62, no. 9, pp. 1421–1432, 2002. [Online]. Available: https://doi.org/10.1006/jpdc.2002.1838
8. P. Malakonakis, M. Smerdis, E. Sotiriades, and A. Dollas, "An FPGA-based Sudoku Solver based on Simulated Annealing methods," in *2009 International Conference on Field-Programmable Technology*, 2009, pp. 522–525. [Online]. Available: https://doi.org/10.1109/FPT.2009.5377608
9. S. McGerty, "Solving Sudoku puzzles with particle swarm optimisation," *Final Report, Macquarie University*, 2009.
10. H. Simonis, "Sudoku as a constraint problem," in *CP Workshop on modeling and reformulating Constraint Satisfaction Problems*, vol. 12, 2005, pp. 13–27.
11. E. C. Chi, "Techniques for Solving Sudoku Puzzles," *arXiv preprint arXiv:1203.2295*, 2012.
12. P. Norvig, "Solving Every Sudoku Puzzle," 2006. [Online]. Available: http://norvig.com/sudoku.html
13. J. F. Crook, "A pencil-and-paper algorithm for solving Sudoku puzzles," *Notices of the AMS*, vol. 56, no. 4, pp. 460–468, 2009.
14. M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT press, 2004.
15. M. Schottlender, "The effect of guess choices on the efficiency of a backtracking algorithm in a Sudoku solver," in *IEEE Long Island Systems, Applications and Technology (LISAT) Conference 2014*, 2014, pp. 1–6. [Online]. Available: https://doi.org/10.1109/LISAT.2014.6845190
16. A. Moraglio and J. Togelius, "Geometric particle swarm optimization for the sudoku puzzle," in *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation*, 2007, pp. 118–125. [Online]. Available: https://doi.org/10.1145/1276958.1276975
17. A. Bhattarai, D. Uprety, P. Pathak, S. N. Shrestha, S. Narkarmi, and S. Sigdel, "A Study Of Sudoku Solving Algorithms: Backtracking and Heuristic," *arXiv preprint arXiv:2507.09708*, 2025.